Book Library System: Docker and Jenkins CI/CD Pipeline

Group Name: Techikaldag
Project Title: Book Library System
Repository: https://github.com/EronSG/BookLibrary-CI-CD
Course/Section: ITE-302
Instructor: Doc. Harwin Mendoza
Date Submitted: October 18, 2026

1. Title Page, Group Members, and Roles

Project Title: Book Library System: A Containerized Application with Automated Testing and Jenkins CI/CD

Group Member	Assigned Role	Responsibilities
Christian Paalan	Project Lead and Documentation	Coordinates project activities, maintains documentation, and monitors completion of requirements.
Neo Yabut	Application Developer	Develops and maintains the Express.js application and its functional modules.
Aleron Gonzales	Containerization and Infrastructure	Configures Docker, Docker Compose, PostgreSQL, and Nginx.
Yahha Arceo	CI/CD and Testing	Configures Jenkins and verifies the pipeline results.
Andrei Jeremy Garcia	CI/CD and Testing	Maintains automated tests and verifies the pipeline results.

2. Project Overview

The Book Library System is a web-based application designed to demonstrate library management functions and modern software deployment practices. It provides modules for managing books, searching for library resources through an external service, and managing users and their borrowing records.

The application uses Express.js for its backend, PostgreSQL for data storage, Docker for containerization, Nginx as a reverse proxy, and Jenkins for Continuous Integration and Continuous Deployment (CI/CD).

The project demonstrates how an application can be packaged into containers, tested automatically, built into a Docker image, and deployed using a repeatable pipeline.

Modules and Responsibilities

Module	Responsibility
Book Management	CRUD operations (create, read, update, delete) for books.
Library Search	Searches library resources through the external OpenLibrary API.
User and Loan Management	Manages users and the borrowing and returning of books.

3. Architecture Diagram and Explanation

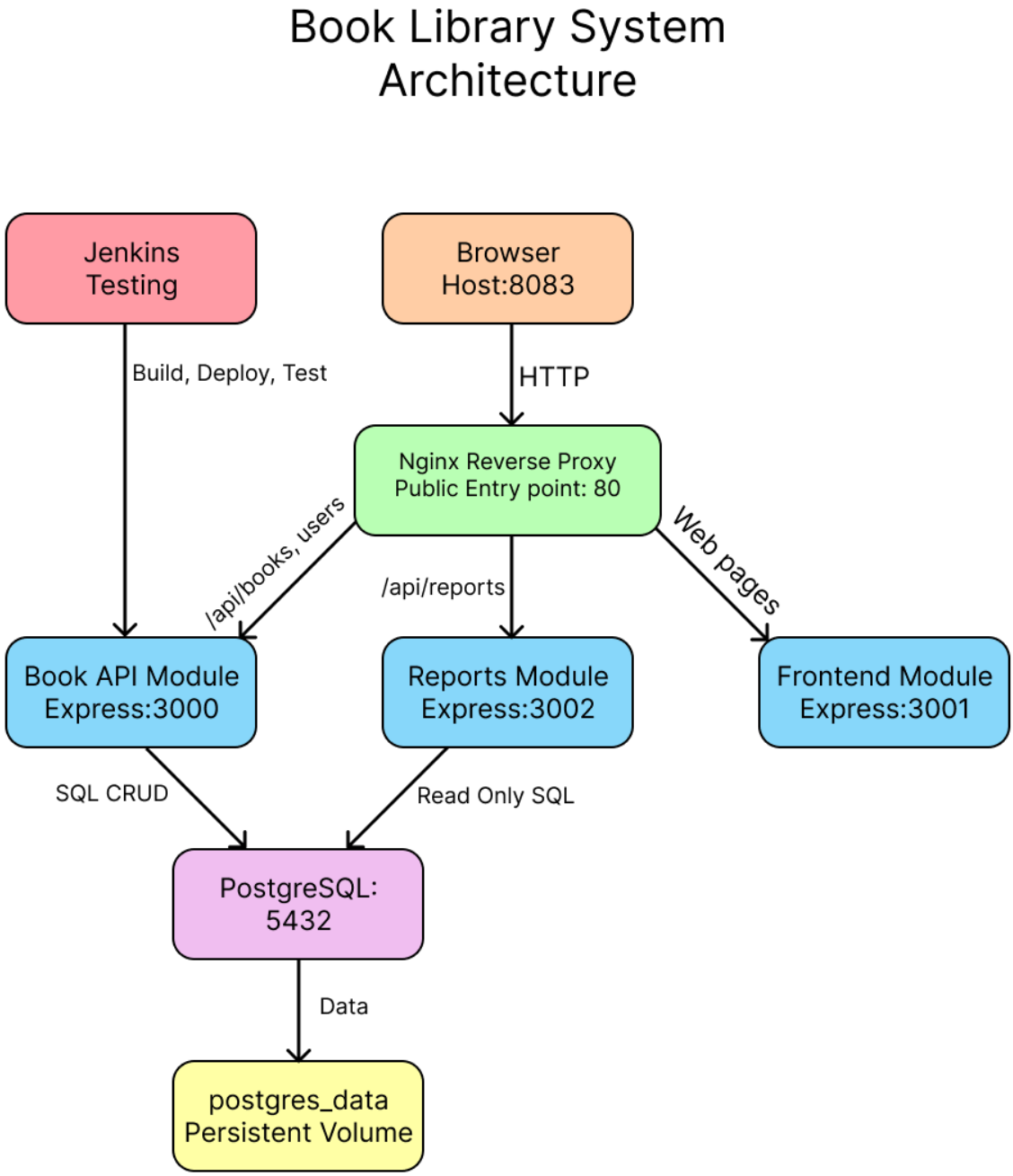


Diagram Explanation: The system architecture is composed of three Docker containerized application modules which consists of Book API, Reports and Frontend. User flow starts with the web browser and the Nginx reverse proxy, forwarding client requests to the appropriate module.

The Book API handles library requests while Reports API retrieves information for library statistics, alongside both of them communicating with the backend with the PostgreSQL Database. Finally, Jenkins handles CI/CD pipeline including source-code checkout, automated testing, image building, deployment, and smoke testing.

Container	Role	Port	Exposed to host?
nginx	Reverse proxy and single entry point	80 (container)	Yes: `NGINX_PORT` maps to 80 (8082 per README, 8083 compose default, 8084 in Jenkins deployment)
web	Express.js application	3000	No, reachable only through Nginx
database	PostgreSQL 17 (Alpine)	5432	No, reachable only by the web container

Network: All services share the default Docker Compose network and reach each other by service name (for example, the app connects to the host `database`).

Volumes:

- `postgres_data` — named volume mounted at `/var/lib/postgresql/data` so database records survive container restarts and rebuilds.
- `./database/init.sql` — mounted read-only into `/docker-entrypoint-initdb.d/` to create the schema on first start.

How a request flows: The browser sends a request to Nginx on the published port. Nginx forwards it to the Express app on port 3000. Express runs the query against PostgreSQL (or calls OpenLibrary for searches) and returns the response back through Nginx. A `/health` endpoint is available for health and smoke checks.

4. Technology Stack and Justification

Technology	Purpose	Justification
Node.js 22 + Express.js	Backend REST API	Lightweight, fast to develop, and well suited to modular REST endpoints.

Technology	Purpose	Justification
PostgreSQL 17	Data storage	Reliable relational database that fits books, users, and loans with relationships.
Docker	Containerization	Same environment on every machine; repeatable builds.
Docker Compose	Multi-container orchestration	Starts the app, database, and proxy together with one command.
Nginx	Reverse proxy	Single public entry point; keeps the app and database off the public network.
Jenkins	CI/CD	Automates testing, image building, deployment, and smoke testing on each run.
Git and GitHub	Version control	Shared repository for team collaboration and Jenkins checkout.
OpenLibrary API	External search	Provides real book data without maintaining our own catalog.

5. Dockerfile Explanations

Application Dockerfile

Instruction	Explanation
`FROM node:22-alpine`	Small Alpine-based Node.js image: faster pulls and a smaller attack surface.
`WORKDIR /app`	Sets the working directory inside the container.
`COPY package*.json ./` then `RUN npm ci --omit=dev`	Dependencies are copied and installed first so Docker caches this layer and only reinstalls when `package.json` changes. `npm ci` gives reproducible installs and `--omit=dev` keeps development tools out of the image.
`COPY . .`	Copies the application source (files in `.dockerignore` are excluded).

Instruction	Explanation
`ENV NODE_ENV=production` and `ENV PORT=3000`	Runs the app in production mode on port 3000.
`EXPOSE 3000`	Documents the application port.
`USER node`	Runs as a non-root user, a security best practice.
`CMD ["node", "server.js"]`	Starts the Express server.

Build style: single-stage build. **Security choices:** minimal Alpine base, non-root user, production-only dependencies, and a `.dockerignore` file.

Other Dockerfiles nginx/Dockerfile

- `nginx/Dockerfile` builds the Nginx reverse-proxy image with the project's configuration.
- `Dockerfile.jenkins` builds the customized Jenkins image used to run the pipeline.

6. Docker Compose Explanation

Service	Image / Build	Key settings
web	Built from `Dockerfile`	`restart: unless-stopped`; environment variables for `NODE_ENV`, `PORT`, `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`; waits for a healthy database.
database	`postgres:17-alpine`	`POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`; named volume `postgres_data`; `init.sql` mounted for schema setup.
nginx	Built from `./nginx`	Depends on `web`; publishes `${NGINX_PORT:-8083}:80`.

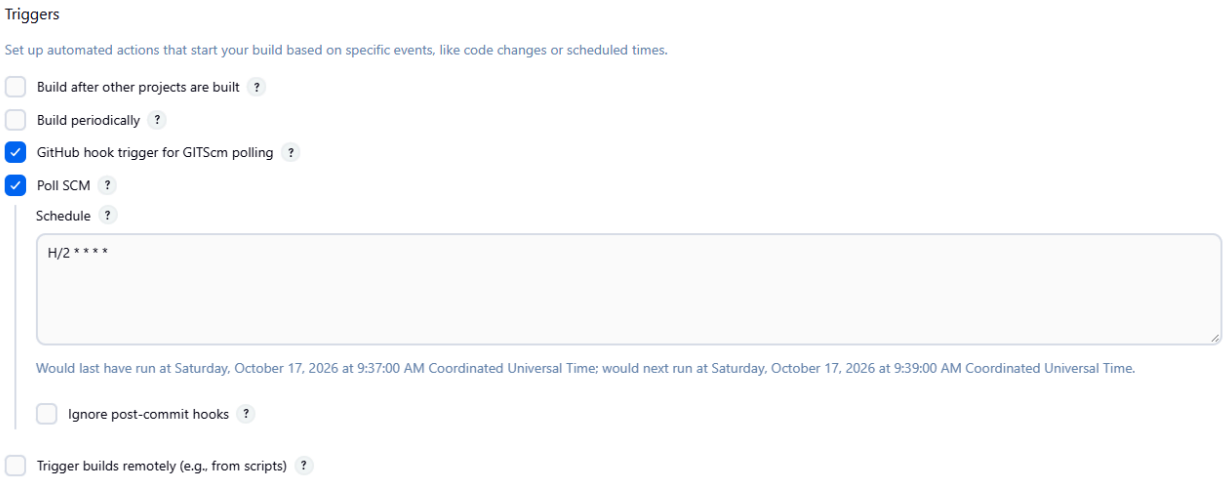
Dependencies: `nginx` starts after `web`, and `web` starts only when `database` reports healthy (`depends_on` with `condition: service_healthy`).

Health check: the database runs `pg_isready -U libraryuser -d library` every 5 seconds, with a 5-second timeout and up to 10 retries. This prevents the app from starting before PostgreSQL is ready.

Environment variables: database credentials and ports are passed as environment variables; `.env.example` documents the expected values without exposing real secrets.

7. Jenkins Setup

- **Installation:** Jenkins runs in a container built from `Dockerfile.jenkins` (see `infra/jenkins`). It is run on the host machine with access to Docker Desktop.
- **Plugins:** The suggested default plugins for Pipeline and Git have been used.
- **Credentials:** The pipeline specifically checks for credential ID `booklibrary-db-password` in testing.
- **Pipeline job:** Pipeline script from SCM, pointing to the GitHub repository, branch `main`, using the `Jenkinsfile` in the repo root.
- **Trigger configuration:** Poll SCM is used with a schedule (`'H/2 * * * *'`) which waits for push and automates building every 2 minutes.



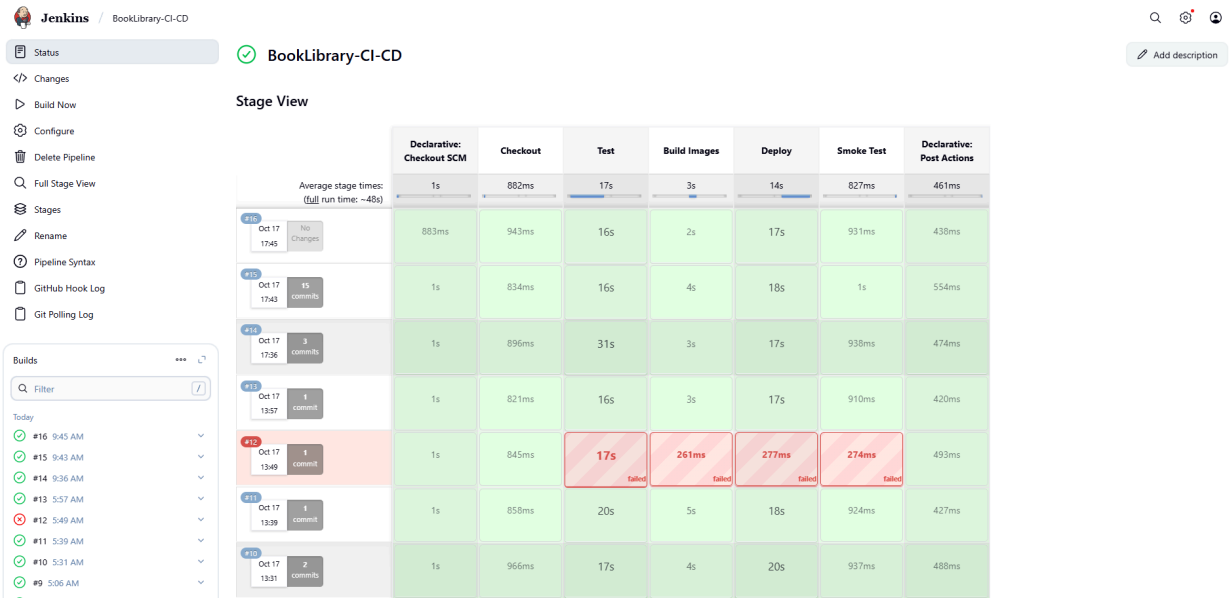


8. Pipeline Explanation

The pipeline is defined in the `Jenkinsfile` and runs on `agent any` with the image name `book-library-system`.

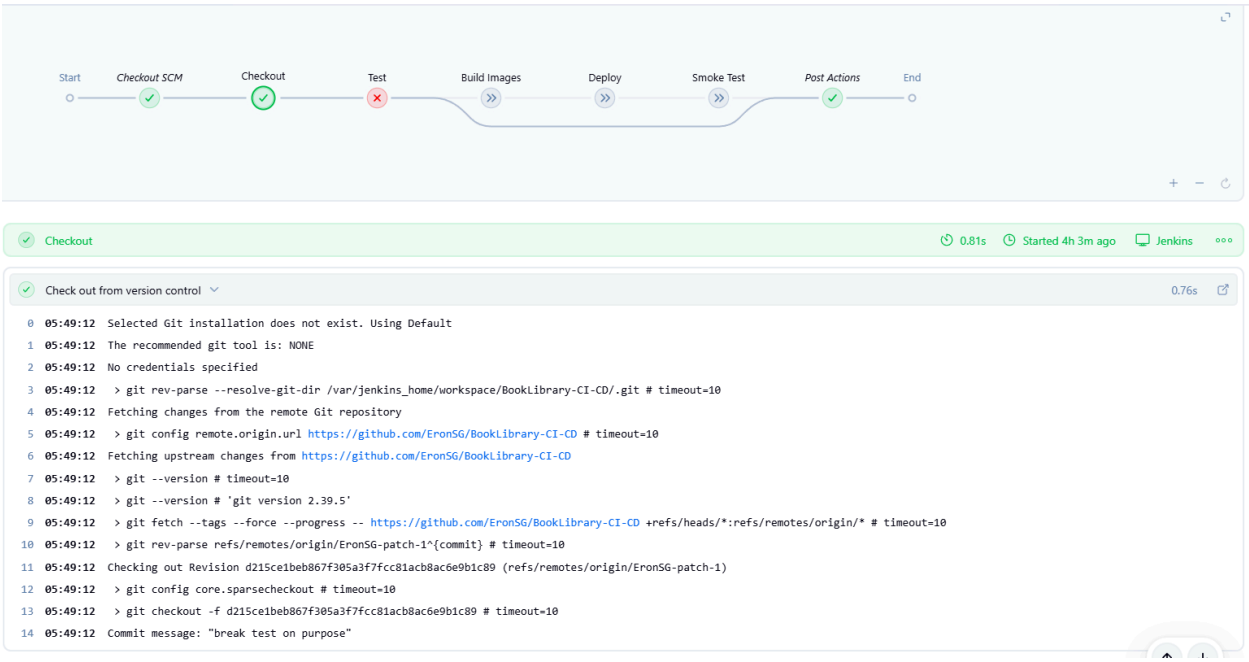
#	Stage	What it does	On failure
1	Checkout	Pulls the latest source code from GitHub (`checkout scm`).	Pipeline stops; nothing is built or deployed.
2	Check Tools	Verifies Node.js, npm, Docker, and Docker Compose are available.	Pipeline stops; the environment problem must be fixed first.
3	Run Tests	Stops old containers, starts the database, then runs `npm install` and `npm test` in a container.	**Test gate:** a failing test fails the build, so no image is built and nothing is deployed.
4	Build Docker Image	Builds `book-library-system:<BUILD_NUMBER>` and tags it `latest`.	Pipeline stops; the running deployment is unchanged.
5	Deploy	Runs `docker compose up -d --build` with `NGINX_PORT=8084`.	Pipeline stops; check the compose logs.
6	Smoke Test	Lists running containers and calls `http://localhost:8084/health` with `curl --fail`.	Build is marked failed because the app is not responding correctly.

Post actions: `docker compose ps` always runs at the end to show container status.



9. Testing Strategy

- **What is tested:** The application's API modules (books, library search, users and loans) through automated tests in the `tests/` folder, run with `npm test`.
- **Where tests run:** Inside the Docker environment against a real PostgreSQL container, so tests match production behavior.
- **Smoke test:** After deployment, `GET /health` confirms the whole stack (Nginx, app) is up.
- **How failures block deployment:** Jenkins runs stages in order and stops at the first failure. Because tests run before Build and Deploy, a failing test prevents any new version from being built or deployed.



10. Rollback Procedure and Data Persistence Proof

Rollback procedure

Both the Compose File and Jenkins support image archiving, so all we have to do is to find the appropriate tag to redeploy to.

10.1 Latest Build and Image.

```
PS D:\BookLibrary> docker images booklibrary-api

IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
booklibrary-api:build-10       197511bdeaf4       248MB         60.2MB
booklibrary-api:build-11       739355d2b45f       248MB         60.2MB
booklibrary-api:build-13       381e1fd51c01       248MB         60.2MB
booklibrary-api:build-14       1633a63d2c05       249MB         60.3MB
booklibrary-api:build-15       ac944c1ddb07       249MB         60.3MB
booklibrary-api:build-16       e5b6d65fdf3a       249MB         60.3MB
booklibrary-api:build-17       27fe59c44872       249MB         60.3MB
booklibrary-api:build-18       0bc476b87038       249MB         60.3MB
booklibrary-api:build-19       3d7aa1eacd7b       249MB         60.5MB
booklibrary-api:build-20       af4b6c9fb152       249MB         60.5MB
booklibrary-api:build-21       cd016668c967       249MB         60.5MB    U
booklibrary-api:build-8        82c99afa48c4       248MB         60.2MB
booklibrary-api:build-9        a47cd1a89095       248MB         60.2MB
booklibrary-api:local          7c9e435404c1       249MB         60.5MB    U
PS D:\BookLibrary>
```

My Library

Books & Search Users & Loans

Module 1: Book Management

Title | Author | Year | Add Book

- PERSISTENCE TEST 2 by PERSISTENCE AUTHOR 2 (1111) Edit Delete
- PERSISTENCE TEST by PERSISTENCE AUTHOR (9999) Edit Delete

Library Reports (Reports Service)

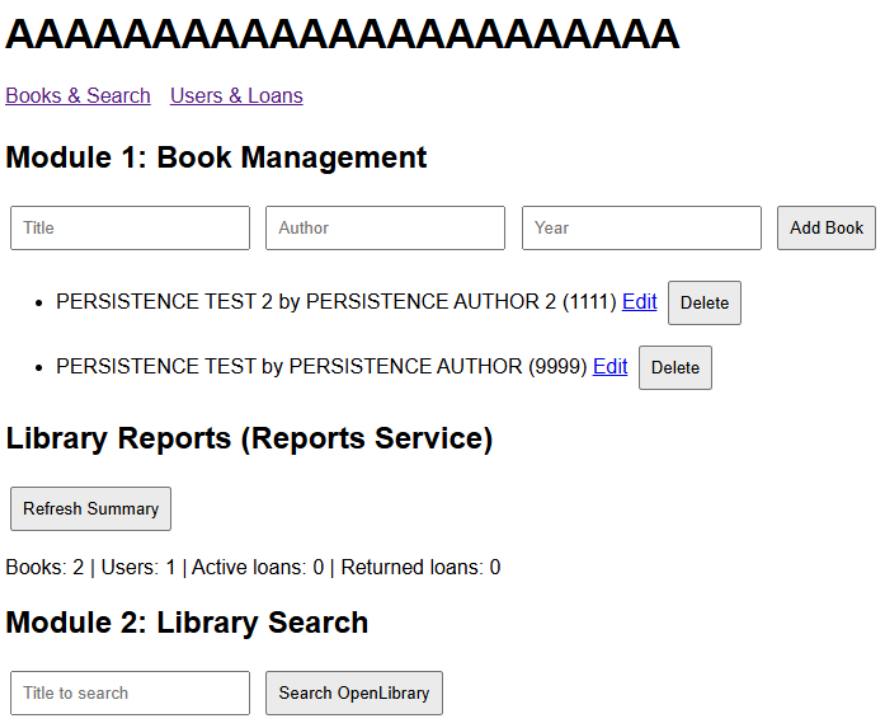
Refresh Summary

Books: 2 | Users: 1 | Active loans: 0 | Returned loans: 0

Module 2: Library Search

Title to search | Search OpenLibrary

10.2 Making a visible change and pushing it



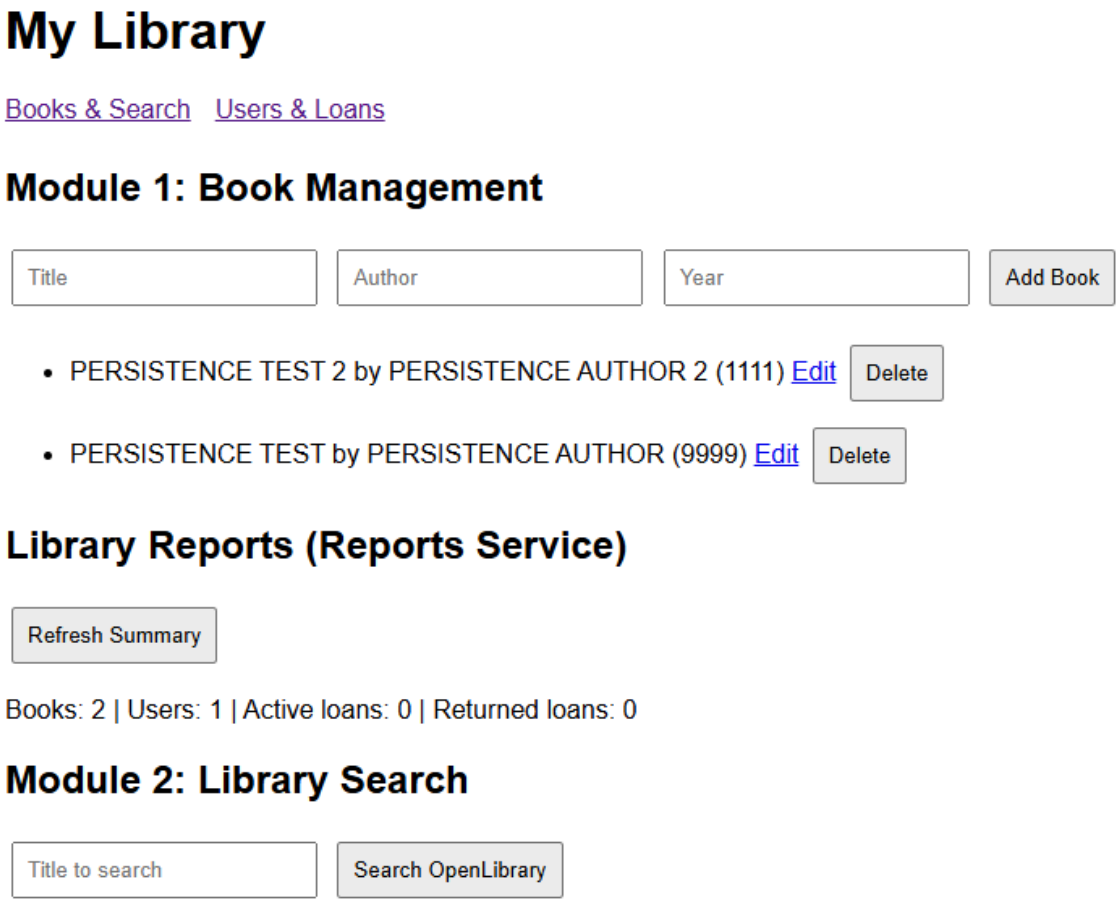
10.3 Verify current image build

```
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
booklibrary-api:build-10       197511bdeaf4       248MB         60.2MB
booklibrary-api:build-11       739355d2b45f       248MB         60.2MB
booklibrary-api:build-13       381e1fd51c01       248MB         60.2MB
booklibrary-api:build-14       1633a63d2c05       249MB         60.3MB
booklibrary-api:build-15       ac944c1ddb07       249MB         60.3MB
booklibrary-api:build-16       e5b6d65fdf3a       249MB         60.3MB
booklibrary-api:build-17       27fe59c44872       249MB         60.3MB
booklibrary-api:build-18       0bc476b87038       249MB         60.3MB
booklibrary-api:build-19       3d7aa1eacd7b       249MB         60.5MB
booklibrary-api:build-20       af4b6c9fb152       249MB         60.5MB
booklibrary-api:build-21       cd016668c967       249MB         60.5MB
booklibrary-api:build-22       2202e638f79d       249MB         60.5MB    U
booklibrary-api:build-8        82c99afa48c4       248MB         60.2MB
booklibrary-api:build-9        a47cd1a89095       248MB         60.2MB
booklibrary-api:local          cfc0dcd973b4       249MB         60.5MB    U
PS D:\BookLibrary>
```

10.4 Rollback to build 21. (Data persists in rebuilding and in every build)

```
PS D:\BookLibrary> $env:IMAGE_TAG = "build-21"
PS D:\BookLibrary> docker compose up -d --no-build --wait
[+] up 5/5
 ✔ Container booklibrary-final-database-1 Healthy
 ✔ Container booklibrary-final-frontend-1 Healthy
 ✔ Container booklibrary-final-book-api-1 Healthy
 ✔ Container booklibrary-final-reports-1  Healthy
 ✔ Container booklibrary-final-proxy-1    Healthy
PS D:\BookLibrary> docker images booklibrary-api

IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
booklibrary-api:build-10       197511bdeaf4       248MB         60.2MB
booklibrary-api:build-11       739355d2b45f       248MB         60.2MB
booklibrary-api:build-13       381e1fd51c01       248MB         60.2MB
booklibrary-api:build-14       1633a63d2c05       249MB         60.3MB
booklibrary-api:build-15       ac944c1ddb07       249MB         60.3MB
booklibrary-api:build-16       e5b6d65fdf3a       249MB         60.3MB
booklibrary-api:build-17       27fe59c44872       249MB         60.3MB
booklibrary-api:build-18       0bc476b87038       249MB         60.3MB
booklibrary-api:build-19       3d7aa1eacd7b       249MB         60.5MB
booklibrary-api:build-20       af4b6c9fb152       249MB         60.5MB
booklibrary-api:build-21       cd016668c967       249MB         60.5MB    U
booklibrary-api:build-22       2202e638f79d       249MB         60.5MB
booklibrary-api:build-8        82c99afa48c4       248MB         60.2MB
booklibrary-api:build-9        a47cd1a89095       248MB         60.2MB
booklibrary-api:local          cfc0dcd973b4       249MB         60.5MB
PS D:\BookLibrary>
```



Data persistence proof

PostgreSQL data is stored in the named volume `postgres_data`, so it survives container removal and redeployment.

11. Problems encountered and how you solved them.

While creating and installing the Book Library System, various kinds of technological problems were faced. Solutions were found for these problems by deploying test techniques and making adjustments to setup.

1. Docker Engine Connection Error

- **Problem:** Docker commands failed because the Docker Desktop Linux engine was not running.
- **Solution:** Docker Desktop was opened manually, and the connection was verified using `docker version`.
- **Result:** Docker commands worked again, and the existing containers could be accessed.

2. PostgreSQL Database Connection Failure

- **Problem:** The application encountered database connection errors, including password authentication failures and connection refusals.
- **Solution:** The database container status, environment variables, database credentials, and application connection settings were checked.
- **Result:** The database health endpoint returned `{"status":"ok","database":"connected"}`.

3. Jenkins Pipeline Execution Issues

- **Problem:** The Jenkins pipeline had execution difficulties since the commands in the pipeline were not suitable for the Windows environment.
- **Solution:** The Jenkins pipeline commands were modified for compatibility with the Jenkins agent on Windows OS. Jenkins was then stopped as per instructor's order not to run Jenkins locally.
- **Result:** Jenkins remained stopped while the Book Library app was functioning normally in the Docker. The Jenkins pipeline requires a valid Jenkins server for running the pipeline.

4. Docker Container Management

- **Problem:** Multiple containers from different Compose projects made it difficult to identify
  To identify which service belonged to the active application

- **Solution:** The containers were inspected using `docker ps`, `docker ps -a`, and `docker compose ps`.
- **Result:** The active Book Library services and their health status were identified without deleting containers or database volumes.

5. Application Health Verification

- **Problem:** It was necessary to confirm that the application remained functional after troubleshooting Docker and Jenkins.
- **Solution:** The health endpoint was tested using `curl.exe --fail http://localhost:8083/health`.
- **Result:** The application returned a successful health response and confirmed its database connection.

12. Who Did What and References

Who Did What

- Christian Paalan – Led the project, coordinated group activities, and prepared and maintained the technical documentation.
- Neo Yabut – Developed and maintained the Express.js application and its functional modules.
- Aleron Gonzales – Configured Docker, Docker Compose, PostgreSQL, and Nginx for containerization and infrastructure.
- Yahha Arceo – Configured Jenkins and tested the CI/CD pipeline.
- Andrei Jeremy Garcia – Maintained automated tests and verified CI/CD pipeline results.

These contributions are based on the assigned roles in your submitted documentation.

References (APA 7th edition)

These are real online sources related to the technologies used in your project. You can use them to support the technical documentation.

1. Docker. (n.d.). Docker Compose. Docker Documentation. https://docs.docker.com/compose/

2. Express.js. (n.d.). Express – Node.js web application framework. https://expressjs.com/

3. Jenkins. (n.d.). Pipeline. Jenkins User Documentation. https://www.jenkins.io/doc/book/pipeline/

4. NGINX. (n.d.). NGINX reverse proxy. NGINX Documentation. https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/

5. Open Library. (n.d.). APIs. https://openlibrary.org/developers/api

6. PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation. https://www.postgresql.org/docs/

7. Techikaldag. (n.d.). Book Library System: Source code repository. GitHub. https://github.com/EronSG/BookLibrary-CI-CD